

CreativeHub Backend - Step by Step Visual Studio Guide

▮ Phase 1: Project Setup in Visual Studio

Step 1: Create New ASP.NET Core Web API Project

1. Open Visual Studio 2022
2. Create New Project → ASP.NET Core Web API
3. **Project Name:** CreativeHubAPI
4. **Solution Name:** CreativeHub
5. **Framework:** .NET 8.0
6. **Authentication Type:** None (we'll implement JWT manually)
7. **Enable OpenAPI Support:** ✓ Checked

Step 2: Install Required NuGet Packages

```
<!-- In CreativeHubAPI.csproj, add these packages -->
<PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0" />
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="8.0.0" />
<PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="7.0.0" />
<PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="8.0.0" />
<PackageReference Include="SixLabors.ImageSharp" Version="3.0.0" />
<PackageReference Include="SixLabors.ImageSharp.Web" Version="3.0.0" />
```

Installation via Package Manager Console:

```
Install-Package Microsoft.EntityFrameworkCore
Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools
Install-Package Microsoft.AspNetCore.Authentication.JwtBearer
Install-Package System.IdentityModel.Tokens.Jwt
Install-Package SixLabors.ImageSharp
```

▮ Phase 2: Create Project Structure

Step 3: Create Folder Structure in Visual Studio

Right-click on project → Add → New Folder for each:

```
CreativeHubAPI/  
├── Controllers/  
├── Models/  
├── DTOs/  
├── Services/  
├── Repositories/  
├── Interfaces/  
├── Data/  
├── Helpers/  
├── Middleware/  
└── Uploads/ (create this folder in wwwroot)
```

▮ Phase 3: Create Data Models

Step 4: Create Models/Role.cs

```
using System.ComponentModel.DataAnnotations;  
  
namespace CreativeHubAPI.Models  
{  
    public class Role  
    {  
        [Key]  
        public string RoleId { get; set; } = Guid.NewGuid().ToString();  
  
        [Required]  
        [StringLength(50)]  
        public string RoleName { get; set; } = string.Empty;  
  
        [StringLength(200)]  
        public string? Description { get; set; }  
  
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;  
  
        // Navigation Properties  
        public virtual ICollection<User> Users { get; set; } = new List<User>();  
    }  
}
```

Step 5: Create Models/User.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CreativeHubAPI.Models
{
    public class User
    {
        [Key]
        public string UserId { get; set; } = Guid.NewGuid().ToString();

        [Required]
        [StringLength(100)]
        public string UserName { get; set; } = string.Empty;

        [Required]
        [EmailAddress]
        [StringLength(200)]
        public string Email { get; set; } = string.Empty;

        [Required]
        public string PasswordHash { get; set; } = string.Empty;

        [StringLength(100)]
        public string? FirstName { get; set; }

        [StringLength(100)]
        public string? LastName { get; set; }

        [StringLength(500)]
        public string? ProfileImagePath { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        // Foreign Keys
        [Required]
        public string RoleId { get; set; } = string.Empty;

        // Navigation Properties
        [ForeignKey("RoleId")]
        public virtual Role Role { get; set; } = null!;

        public virtual ICollection<Asset> CreatedAssets { get; set; } = new List<Asset>();
        public virtual ICollection<Project> CreatedProjects { get; set; } = new List<Project>();
        public virtual ICollection<UserProject> UserProjects { get; set; } = new List<UserProject>();
    }
}
```

Step 6: Create Models/Category.cs

```
using System.ComponentModel.DataAnnotations;

namespace CreativeHubAPI.Models
{
    public class Category
    {
        [Key]
        public string CategoryId { get; set; } = Guid.NewGuid().ToString();

        [Required]
        [StringLength(100)]
        public string Name { get; set; } = string.Empty;

        [StringLength(500)]
        public string? Description { get; set; }

        [StringLength(500)]
        public string? IconPath { get; set; }

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        // Navigation Properties
        public virtual ICollection<Asset> Assets { get; set; } = new List<Asset>();
    }
}
```

Step 7: Create Models/Project.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CreativeHubAPI.Models
{
    public class Project
    {
        [Key]
        public string ProjectId { get; set; } = Guid.NewGuid().ToString();

        [Required]
        [StringLength(200)]
        public string Name { get; set; } = string.Empty;

        [StringLength(1000)]
        public string? Description { get; set; }

        [StringLength(200)]
        public string? ClientName { get; set; }

        public DateTime? StartDate { get; set; }
        public DateTime? EndDate { get; set; }

        [StringLength(500)]
    }
```

```

        public string? ThumbnailPath { get; set; }

        [StringLength(50)]
        public string Status { get; set; } = "Active"; // Active, Completed, OnHold

        public bool IsPublic { get; set; } = false;

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        // Foreign Keys
        [Required]
        public string CreatedBy { get; set; } = string.Empty;

        // Navigation Properties
        [ForeignKey("CreatedBy")]
        public virtual User Creator { get; set; } = null!;

        public virtual ICollection<Asset> Assets { get; set; } = new List<Asset>();
        public virtual ICollection<UserProject> UserProjects { get; set; } = new List<UserProject>();
    }
}

```

Step 8: Create Models/Asset.cs

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CreativeHubAPI.Models
{
    public class Asset
    {
        [Key]
        public string AssetId { get; set; } = Guid.NewGuid().ToString();

        [Required]
        [StringLength(300)]
        public string Title { get; set; } = string.Empty;

        [StringLength(1000)]
        public string? Description { get; set; }

        public long FileSize { get; set; }

        [StringLength(100)]
        public string? Dimensions { get; set; } // e.g., "1920x1080"

        [StringLength(200)]
        public string? ColorPalette { get; set; } // JSON array of hex colors

        public bool IsPublic { get; set; } = false;

        public int DownloadCount { get; set; } = 0;
        public int LikeCount { get; set; } = 0;

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    }
}

```

```

        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;

        // Foreign Keys
        [Required]
        public string CategoryId { get; set; } = string.Empty;

        public string? ProjectId { get; set; }

        [Required]
        public string CreatedBy { get; set; } = string.Empty;

        // Navigation Properties
        [ForeignKey("CategoryId")]
        public virtual Category Category { get; set; } = null!;

        [ForeignKey("ProjectId")]
        public virtual Project? Project { get; set; }

        [ForeignKey("CreatedBy")]
        public virtual User Creator { get; set; } = null!;

        public virtual ICollection<AssetFile> Files { get; set; } = new List<AssetFile>();
        public virtual ICollection<AssetTag> AssetTags { get; set; } = new List<AssetTag>();
    }
}

```

Step 9: Create Models/AssetFile.cs

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CreativeHubAPI.Models
{
    public class AssetFile
    {
        [Key]
        public string FileId { get; set; } = Guid.NewGuid().ToString();

        [Required]
        [StringLength(300)]
        public string FileName { get; set; } = string.Empty;

        [Required]
        [StringLength(500)]
        public string FilePath { get; set; } = string.Empty;

        [Required]
        [StringLength(10)]
        public string FileType { get; set; } = string.Empty; // jpg, png, pdf, etc.

        [Required]
        [StringLength(100)]
        public string MimeType { get; set; } = string.Empty;

        public long FileSize { get; set; }
    }
}

```

```

        public bool IsOriginal { get; set; } = true;

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        // Foreign Keys
        [Required]
        public string AssetId { get; set; } = string.Empty;

        // Navigation Properties
        [ForeignKey("AssetId")]
        public virtual Asset Asset { get; set; } = null!;
    }
}

```

Step 10: Create Models/Tag.cs and AssetTag.cs

```

// Models/Tag.cs
using System.ComponentModel.DataAnnotations;

namespace CreativeHubAPI.Models
{
    public class Tag
    {
        [Key]
        public string TagId { get; set; } = Guid.NewGuid().ToString();

        [Required]
        [StringLength(50)]
        public string TagName { get; set; } = string.Empty;

        [StringLength(7)]
        public string? Color { get; set; } = "#007bff"; // Hex color code

        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

        // Navigation Properties
        public virtual ICollection<AssetTag> AssetTags { get; set; } = new List<AssetTag>();
    }
}

// Models/AssetTag.cs
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CreativeHubAPI.Models
{
    public class AssetTag
    {
        [Key]
        public string AssetTagId { get; set; } = Guid.NewGuid().ToString();

        // Foreign Keys
        [Required]
        public string AssetId { get; set; } = string.Empty;
    }
}

```

```

        [Required]
        public string TagId { get; set; } = string.Empty;

        // Navigation Properties
        [ForeignKey("AssetId")]
        public virtual Asset Asset { get; set; } = null!;

        [ForeignKey("TagId")]
        public virtual Tag Tag { get; set; } = null!;
    }
}

```

Step 11: Create Models/UserProject.cs

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace CreativeHubAPI.Models
{
    public class UserProject
    {
        [Key]
        public string UserProjectId { get; set; } = Guid.NewGuid().ToString();

        [StringLength(50)]
        public string Role { get; set; } = "Viewer"; // Owner, Collaborator, Viewer

        public DateTime JoinedAt { get; set; } = DateTime.UtcNow;

        // Foreign Keys
        [Required]
        public string UserId { get; set; } = string.Empty;

        [Required]
        public string ProjectId { get; set; } = string.Empty;

        // Navigation Properties
        [ForeignKey("UserId")]
        public virtual User User { get; set; } = null!;

        [ForeignKey("ProjectId")]
        public virtual Project Project { get; set; } = null!;
    }
}

```

Phase 4: Create Database Context

Step 12: Create Data/CreativeHubContext.cs

```
using Microsoft.EntityFrameworkCore;
using CreativeHubAPI.Models;

namespace CreativeHubAPI.Data
{
    public class CreativeHubContext : DbContext
    {
        public CreativeHubContext(DbContextOptions<CreativeHubContext> options) : base(options)
        {
        }

        // DbSet
        public DbSet<Role> Roles { get; set; }
        public DbSet<User> Users { get; set; }
        public DbSet<Category> Categories { get; set; }
        public DbSet<Project> Projects { get; set; }
        public DbSet<Asset> Assets { get; set; }
        public DbSet<AssetFile> AssetFiles { get; set; }
        public DbSet<Tag> Tags { get; set; }
        public DbSet<AssetTag> AssetTags { get; set; }
        public DbSet<UserProject> UserProjects { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Configure relationships
            ConfigureRelationships(modelBuilder);

            // Seed initial data
            SeedData(modelBuilder);
        }

        private void ConfigureRelationships(ModelBuilder modelBuilder)
        {
            // User -> Role (Many to One)
            modelBuilder.Entity<User>()
                .HasOne(u => u.Role)
                .WithMany(r => r.Users)
                .HasForeignKey(u => u.RoleId)
                .OnDelete(DeleteBehavior.Restrict);

            // Asset -> Category (Many to One)
            modelBuilder.Entity<Asset>()
                .HasOne(a => a.Category)
                .WithMany(c => c.Assets)
                .HasForeignKey(a => a.CategoryId)
                .OnDelete(DeleteBehavior.Restrict);

            // Asset -> Project (Many to One, Optional)
            modelBuilder.Entity<Asset>()
                .HasOne(a => a.Project)
                .WithMany(p => p.Assets)
                .HasForeignKey(a => a.ProjectId)
        }
    }
}
```

```

        .onDelete(DeleteBehavior.SetNull);

// Asset -> User (Many to One, Creator)
modelBuilder.Entity<Asset>()
    .HasOne(a => a.Creator)
    .WithMany(u => u.CreatedAssets)
    .HasForeignKey(a => a.CreatedBy)
    .onDelete(DeleteBehavior.Restrict);

// AssetFile -> Asset (Many to One)
modelBuilder.Entity<AssetFile>()
    .HasOne(af => af.Asset)
    .WithMany(a => a.Files)
    .HasForeignKey(af => af.AssetId)
    .onDelete(DeleteBehavior.Cascade);

// Project -> User (Many to One, Creator)
modelBuilder.Entity<Project>()
    .HasOne(p => p.Creator)
    .WithMany(u => u.CreatedProjects)
    .HasForeignKey(p => p.CreatedBy)
    .onDelete(DeleteBehavior.Restrict);

// UserProject -> User (Many to One)
modelBuilder.Entity<UserProject>()
    .HasOne(up => up.User)
    .WithMany(u => u.UserProjects)
    .HasForeignKey(up => up.UserId)
    .onDelete(DeleteBehavior.Cascade);

// UserProject -> Project (Many to One)
modelBuilder.Entity<UserProject>()
    .HasOne(up => up.Project)
    .WithMany(p => p.UserProjects)
    .HasForeignKey(up => up.ProjectId)
    .onDelete(DeleteBehavior.Cascade);

// AssetTag -> Asset (Many to One)
modelBuilder.Entity<AssetTag>()
    .HasOne(at => at.Asset)
    .WithMany(a => a.AssetTags)
    .HasForeignKey(at => at.AssetId)
    .onDelete(DeleteBehavior.Cascade);

// AssetTag -> Tag (Many to One)
modelBuilder.Entity<AssetTag>()
    .HasOne(at => at.Tag)
    .WithMany(t => t.AssetTags)
    .HasForeignKey(at => at.TagId)
    .onDelete(DeleteBehavior.Cascade);

// Unique constraints
modelBuilder.Entity<User>()
    .HasIndex(u => u.UserName)
    .IsUnique();

```

```

        modelBuilder.Entity<User>()
            .HasIndex(u => u.Email)
            .IsUnique();

        modelBuilder.Entity<Tag>()
            .HasIndex(t => t.TagName)
            .IsUnique();
    }

    private void SeedData(ModelBuilder modelBuilder)
    {
        // Seed Roles
        var roles = new[]
        {
            new Role { RoleId = "R001", RoleName = "Admin", Description = "Full system access", IsActive = true },
            new Role { RoleId = "R002", RoleName = "Designer", Description = "Create and edit content", IsActive = true },
            new Role { RoleId = "R003", RoleName = "Client", Description = "View assigned content", IsActive = true },
            new Role { RoleId = "R004", RoleName = "Viewer", Description = "View public content", IsActive = true },
        };
        modelBuilder.Entity<Role>().HasData(roles);

        // Seed Categories
        var categories = new[]
        {
            new Category { CategoryId = "CAT001", Name = "Portfolio", Description = "Showcase your work", IsActive = true },
            new Category { CategoryId = "CAT002", Name = "Client Work", Description = "Examples of client projects", IsActive = true },
            new Category { CategoryId = "CAT003", Name = "Templates", Description = "Design templates for sale", IsActive = true },
            new Category { CategoryId = "CAT004", Name = "Logos", Description = "Branding and logo design", IsActive = true },
            new Category { CategoryId = "CAT005", Name = "Stock Photos", Description = "High-quality stock photography", IsActive = true },
            new Category { CategoryId = "CAT006", Name = "Videos", Description = "Creative video content", IsActive = true },
        };
        modelBuilder.Entity<Category>().HasData(categories);

        // Seed Admin User
        modelBuilder.Entity<User>().HasData(new User
        {
            UserId = "U001",
            UserName = "admin",
            Email = "admin@creativehub.com",
            PasswordHash = BCrypt.Net.BCrypt.HashPassword("Admin123!"),
            FirstName = "System",
            LastName = "Administrator",
            RoleId = "R001",
            IsActive = true
        });

        // Seed Sample Designer
        modelBuilder.Entity<User>().HasData(new User
        {
            UserId = "U002",
            UserName = "designer1",
            Email = "designer@creativehub.com",
            PasswordHash = BCrypt.Net.BCrypt.HashPassword("Designer123!"),
            FirstName = "John",
            LastName = "Designer",
            RoleId = "R002",
        });
    }
}

```

```

        IsActive = true
    });

    // Seed Tags
    var tags = new[]
    {
        new Tag { TagId = "T001", TagName = "Web Design", Color = "#007bff" },
        new Tag { TagId = "T002", TagName = "Mobile App", Color = "#28a745" },
        new Tag { TagId = "T003", TagName = "Branding", Color = "#dc3545" },
        new Tag { TagId = "T004", TagName = "Photography", Color = "#ffc107" },
        new Tag { TagId = "T005", TagName = "Video", Color = "#6f42c1" }
    };
    modelBuilder.Entity<Tag>().HasData(tags);
}
}
}

```

Note: Add BCrypt.Net-Next NuGet package for password hashing:

```
Install-Package BCrypt.Net-Next
```

Continue to Part 2 for DTOs, Services, and Controllers implementation...